

Thread-Safe Singleton Design Pattern in C# with Examples

In this article, I will discuss **How to Implement a Thread-Safe Singleton Design Pattern in C#** with Examples. Please read our previous article discussing [why we must create the Singleton Class as Sealed in C#](#) with Examples. As of now, the way we have implemented the Singleton Design Pattern is not Thread Safe in a Multithread Environment. As part of this article, we will discuss the following pointers in detail.

1. [What is Thread-Safe in Singleton Design Pattern?](#)
2. [What is Lazy Initialization in the Singleton Design Pattern?](#)
3. [How Do We Implement the Singleton Design Pattern in a Thread-Safe Manner in C#?](#)
4. [Implementation of Thread-Safe Singleton Design Pattern using Locks.](#)
5. [Double-Check Locking Approach to Implement Thread-Safe Singleton Design Pattern in C#.](#)
6. [Thread-Safe Singleton Design Pattern in C# using Lazy<T>.](#)

Thread-Safe in Singleton Design Pattern in C#.

The Thread-Safe Singleton Design Pattern in C# ensures that a class has only one instance and provides a global point of access to it while also ensuring that the instance is created safely in a multi-threaded environment. In a multi-threaded application, it is important to ensure that the Singleton Design Pattern prevents multiple threads from creating the Singleton instance more than once.

Issues with Non-Thread-Safe Singleton

In a non-thread-safe implementation, if multiple threads access the Singleton instance simultaneously at the point when the instance has not yet been created, each thread could end up creating a new instance. This violates the core principle of the Singleton pattern, which is only to allow one instance of the class.

Let us first see the problem we face in a multithread environment if the Singleton Class is not Thread-Safe. So, create a class file with the name **Singleton.cs** and copy and paste the following code. The following code example is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        private static Singleton Instance = null;
        //The following Static Method is going to return the Singleton
Instance
        public static Singleton GetInstance()
        {
            //This is not thread-safe
            if (Instance == null)
            {
                Instance = new Singleton();
            }
            //Return the Singleton Instance
            return Instance;
        }

        private Singleton()
```

```

    {
        //Each Time the Constructor is called, increment the Counter
        value by 1
        Counter++;
        Console.WriteLine("Counter Value " + Counter.ToString());
    }
    //The following method can be accessed from outside of the class by
    using the Singleton Instance
    public void PrintDetails(string message)
    {
        Console.WriteLine(message);
    }
}

```

The way we have created the above Singleton Class is not Thread-Safe in a Multithreaded environment. That means two threads may check the **if (instance == null)** condition at the same time, and it is also possible that both threads found the condition true, resulting in two instances of the Singleton Class being created.

Consuming the Non-Thread Safe Singleton Design Pattern:

Let's modify the Main method of the Program class as follows to use multithread programming. As you can see in the below code, we are using **Parallel.Invoke** method and call PrintTeacherDetails and PrintStudentDetails methods parallel. This will create two threads. Further, if you notice, we are calling the Static GetInstance() method of the Singleton class in each method. That means two different threads are now trying to call the same Static GetInstance() method of the Singleton class. Let us run the application, see the output, and check how many times the constructor of the Singleton class is executed.

```

using System;
using System.Threading.Tasks;
namespace SingletonDemo
{
    class Program
    {
        //Example to Understand Thraed-Safe Problem with Singleton Design
        Pattern
        //When use in a Multithreaded Environment
        static void Main(string[] args)
        {
            //The following Code will Invoke both methods Parallely using
            two different Threads
            Parallel.Invoke(
                //Let us Assume PrintTeacherDetails method is Invoked by
                Thread-1
                () => PrintTeacherDetails(),
                //Let us Assume PrintStudentDetails method is Invoked by
                Thread-2
                () => PrintStudentdetails()
            );
            Console.ReadLine();
        }
        private static void PrintTeacherDetails()
        {
            //Thread-1 Calling the GetInstance() Method of the Singleton
            class

```

```

        Singleton fromTeacher = Singleton.GetInstance();
        fromTeacher.PrintDetails("From Teacher");
    }
    private static void PrintStudentDetails()
    {
        //At the same time, Thread-2 also Calling the GetInstance()
Method of the Singleton Class
        Singleton fromStudent = Singleton.GetInstance();
        fromStudent.PrintDetails("From Student");
    }
}
}

```

Here, within the main method, we use **Parallel.Invoke** method to invoke Multiple Methods parallelly; This concept is introduced in .NET Framework 4.0. If you are new to parallel programming, please read our [Parallel Programming in C#](#) with Real-Time examples article.

So, in our example, we are using the **Parallel.Invoke** method to access the GetInstance Method parallelly. That means, at the same time, multiple threads are accessing the GetInstance Method. The above code is not Thread-Safe because the way we have written it here, two different threads can evaluate the condition if (instance == null) simultaneously, and both threads find it true. They both will create instances that violate the singleton design pattern. So, now run the application, and it will give you the following output.

```

Counter Value 2
From Student
Counter Value 1
From Teacher

```

The above output clearly shows that the counter value has incremented to 2, which proves that the constructor of the Singleton class is executed two times; as a result, two instances of the singleton class have been created in this case. So, if the Singleton class is not Thread-Safe, we may create multiple instances of the Singleton class in a Multithread Environment.

What is Lazy Initialization in Singleton Design Pattern in C#?

In our example, the Singleton Instance will not be created until and unless we invoke the GetInstance Method of the Singleton Class. That means only the Singleton Instance will be created when we invoke the GetInstance Method of the Singleton class. This Delay in creating the Singleton Instance is called Lazy Initialization in the Singleton Design Pattern.

The Lazy Initialization, i.e., the on-demand object creation of the Singleton Class, works fine when we invoke the GetInstance Method in a Single-Threaded environment. However, it will not work as expected in a multi-threaded environment. In a Multi-Thread environment, the lazy initialization may create multiple instances of the singleton class when multiple threads invoke the GetInstance Method parallelly at the same time, as we just discussed.

How Do We Implement a Thread-Safe Singleton Design Pattern in C#?

There are many ways to Implement the Thread-Safe Singleton Design Pattern in C#. They are as follows:

1. Thread-Safety Singleton Implementation using Lock.
2. Implementing Thread-Safety Singleton Design Pattern using Double-Check Locking.
3. Using Eager Loading to Implement Thread-Safety Singleton Design Pattern
4. Using Lazy<T> Generic Class to Implement Lazy Loading in Singleton Design Pattern.

We will discuss all the above mechanisms to Implement the Thread-Safe Singleton Design Pattern in C#. In this article, we will discuss how to use Locks and Double-Check Locking Mechanisms to implement the Thread-Safe

Singleton Design Pattern in C#, which will create only one instance of the Singleton Class in a Multithread environment. The other [Two Mechanisms](#) will be discussed in our next article.

Implementation of Thread-safe Singleton Design Pattern in C# using Locks.

Let us discuss using locks to implement the Thread-Safe Singleton Design Pattern in C#. Locks are the best option for handling the singleton instance in a multithread environment. Using locks, we can synchronize the method so that only one thread can access it at any given point in time.

For a better understanding, please modify the Singleton Class as follows. In the following code, we lock the shared resource using the lock object and check whether the instance is created. If the instance is already created, we return that instance; else, we will create the instance and then return that instance. The most important point is that as long as one thread locks the resource, no other thread can access the resource. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        private static Singleton Instance = null;
        //To use the lock, we need to create one variable
        private static readonly object Instancelock = new object();
        //The following Static Method is going to return the Singleton
Instance
        public static Singleton GetInstance()
        {
            //This is thread-safe
            //As long as one thread locks the resource, no other thread can
access the resource
            //As long as one thread enters into the Critical Section,
            //no other threads are allowed to enter the critical section
            lock (Instancelock)
            { //Critical Section Start
                if (Instance == null)
                {
                    Instance = new Singleton();
                }
            } //Critical Section End
            //Once the thread releases the lock, the other thread allows
entering into the critical section
            //But only one thread is allowed to enter the critical section
            //Return the Singleton Instance
            return Instance;
        }

        private Singleton()
        {
            //Each Time the Constructor is called, increment the Counter
value by 1
            Counter++;
        }
    }
}
```

```

        Console.WriteLine("Counter Value " + Counter.ToString());
    }
    //The following method can be accessed from outside of the class by
using the Singleton Instance
    public void PrintDetails(string message)
    {
        Console.WriteLine(message);
    }
}
}

```

With the above changes in place, now run the application, and you will get the following output.

```

Counter Value 1
From Teacher
From Student

```

The above code implementation using lock solves the Singleton Design Pattern Thread-Safe issues in a multithreading environment. However, it slows down your application as only one thread can access the GetInstance method at a time. We can overcome the above problem by using the **Double-Checked Locking** mechanism.

Double Checked Locking Approach to Implement Thread-Safe Singleton Design Pattern in C#.

The double-check locking approach is a common technique used to reduce the overhead of acquiring a lock every time the singleton instance needs to be accessed while still ensuring that the instance is created lazily and only once. It involves two checks (double check):

- The first check is to see if the instance is null before acquiring the lock.
- The second check is performed after the lock is acquired to ensure that no other thread created the instance in the meantime.

Please modify the Singleton class as follows. The following example code is self-explained, so please go through the comment lines to better understand.

```

using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        private static Singleton Instance = null;
        //To use the lock, we need to create one variable
        private static readonly object Instancelock = new object();
        //The following Static Method is going to return the Singleton
Instance
        public static Singleton GetInstance()
        {
            //This is thread-Safe
            if (Instance == null)
            {

```

```

        //As long as one thread locks the resource, no other thread
can access the resource
        //As long as one thread enters into the Critical Section,
        //no other threads are allowed to enter the critical
section
        lock (InstanceLock)
        { //Critical Section Start
            if (Instance == null)
            {
                Instance = new Singleton();
            }
        } //Critical Section End
        //Once the thread releases the lock, the other thread
allows entering into the critical section
        //But only one thread is allowed to enter the critical
section
    }

    //Return the Singleton Instance
    return Instance;
}

private Singleton()
{
    //Each Time the Constructor is called, increment the Counter
value by 1
    Counter++;
    Console.WriteLine("Counter Value " + Counter.ToString());
}
//The following method can be accessed from outside of the class by
using the Singleton Instance
public void PrintDetails(string message)
{
    Console.WriteLine(message);
}
}
}

```

Explanation of Double-Check Locking

- **First Check:** The first `if (Instance == null)` check avoids locking each time the instance needs to be accessed, which improves performance once the instance is initialized.
- **Lock:** The lock block provides thread safety, ensuring that only one thread can create the instance of the Singleton at any given time. This step is important to prevent race conditions where two or more threads try to create the singleton instance simultaneously.
- **Second Check:** The second `if (_instance == null)` check inside the lock block is essential because it re-checks the instance state after acquiring the lock. Multiple threads might have been waiting for the lock when the instance was null. More than one instance could be created without the second check, which violates the Singleton pattern.

With the above changes in place, now run the application, and you will see the output as expected, as shown in the below image.

```
Counter Value 1
From Teacher
From Student
```

Note: While double-check locking is efficient, the initial access (or accesses in a heavily concurrent scenario) might involve some overhead due to locking. However, once the instance is created, the overhead of checking the instance state is minimal.

Thread-Safe Singleton Design Pattern Implementation using Lazy<T> in C#:

Using the Lazy<T> type in C# is one of the most effective ways to implement a thread-safe Singleton design pattern. The Lazy<T> type handles lazy initialization and thread safety, ensuring that the Singleton instance is created only once, even in multi-threaded environments. This approach simplifies the code and eliminates the need for explicit locking or double-check locking patterns. The following are the key features of the Lazy<T> type:

- **Thread Safety:** By default, Lazy<T> is thread-safe. It uses locks internally to ensure that the value is initialized only once.
- **Lazy Initialization:** The instance is not created until it is actually needed, which can improve the application's startup time and overall memory usage efficiency.
- **Simple and Clean:** The code is cleaner and easier to maintain, as the complexity of thread management is abstracted away by the Lazy<T> type.

For a better understanding, please modify the Singleton class as follows to implement a thread-safe Singleton Pattern using Lazy<T>. The following example code is self-explained, so please go through the comment lines to better understand.

```
using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
of the class is created
        private static int Counter = 0;
        // The private static instance ensures lazy initialization.
        private static readonly Lazy<Singleton> instance = new
Lazy<Singleton>(() => new Singleton());
        // Private constructor to prevent direct instantiation.
        private Singleton()
        {
            // Initialization code here
            //Each Time the Constructor is called, increment the Counter
value by 1
            Counter++;
            Console.WriteLine("Counter Value " + Counter.ToString());
        }
        //The following Static Method is going to return the Singleton
Instance
        public static Singleton GetInstance()
        {
            return instance.Value;
        }
        //The following method can be accessed from outside of the class by
using the Singleton Instance
        public void PrintDetails(string message)
        {
```

```

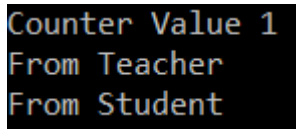
        Console.WriteLine(message);
    }
}

```

Explanation:

- **Lazy<T> Initialization:** The Singleton instance is wrapped in a Lazy<T> object, which is initialized with a delegate (() => new Singleton()) that points to the private constructor of the Singleton. The Singleton instance is created the first time **Lazy<T>.Value** is accessed.
- **Thread Safety:** The Lazy<T> class by default ensures thread safety, which means no matter how many threads try to access the **Singleton.GetInstance()** method simultaneously, the Singleton constructor is executed only once, and all threads will receive the same instance.
- **Performance:** This method is very efficient because after the instance is created, accessing the **Singleton.GetInstance()** method does not incur any lock-checking overhead typically associated with other thread-safe approaches.

With the above changes in place, now run the application, and you will see the output as expected, as shown in the below image.



```

Counter Value 1
From Teacher
From Student

```

In the next article, I will discuss the [Lazy vs Eager Loading in Singleton Design Pattern in C#](#) with Examples. In this article, I explain **How to Implement a Thread-Safe Singleton Design Pattern in C#** using the Lock and Double-Check Locking Mechanism in C# with Examples. I hope you enjoy this How to Implement Thread-Safe Singleton Design Pattern in C# with Examples article. Please give your feedback, questions, and suggestions about this article.